

RFID and Cryptographic Subsystem

Group 8

K Ramya

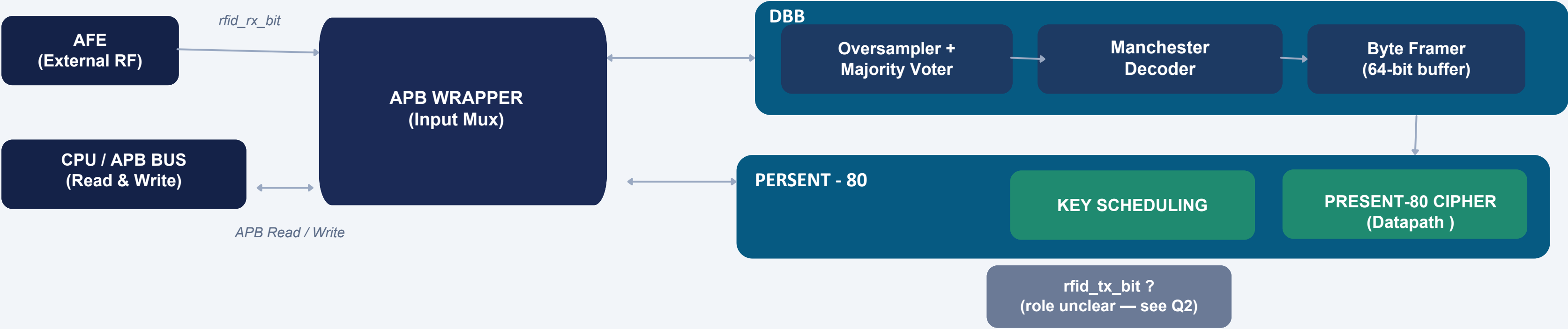
Kummetha Manvitha

K. Mahitha

Naga Jeshwanth Sreeram

Vancha Avinash Reddy

Full Subsystem Data Flow — All Modes



MODE-BY-MODE DATA FLOW

Mode 0 — Live — *Wrapper routes live RX pipeline output to DATA_OUT*



Mode 1 — DBB Loopback — *Wrapper feeds APB writes into DBB (bypasses AFE); DBB output → DATA_OUT*



Mode 2 — Crypto Loopback — *Wrapper feeds APB writes straight into Cipher (bypasses DBB); Cipher output → DATA_OUT*



Open Questions Before Implementation

1

Is the PRESENT-80 key always fixed for a session, or can it be updated at runtime? If it can change, how does the wrapper (or CPU) know the key has been updated — a dedicated status bit, or re-writing KEY_0–2 always triggers a reload?

0x08	ADDR_KEY_0	W/O	Lower 32 bits of	PRESENT-80 Key.
0x0C	ADDR_KEY_1	W/O	Middle 32 bits of	PRESENT-80 Key.
0x10	ADDR_KEY_2	W/O	Upper 16 bits of	PRESENT-80 Key.

2

What is the actual role of rfid_tx_bit? Do we need to transmit data out on it (e.g., echoing DATA_OUT or a loopback response), or is it left unconnected / driven low for this phase of the project?

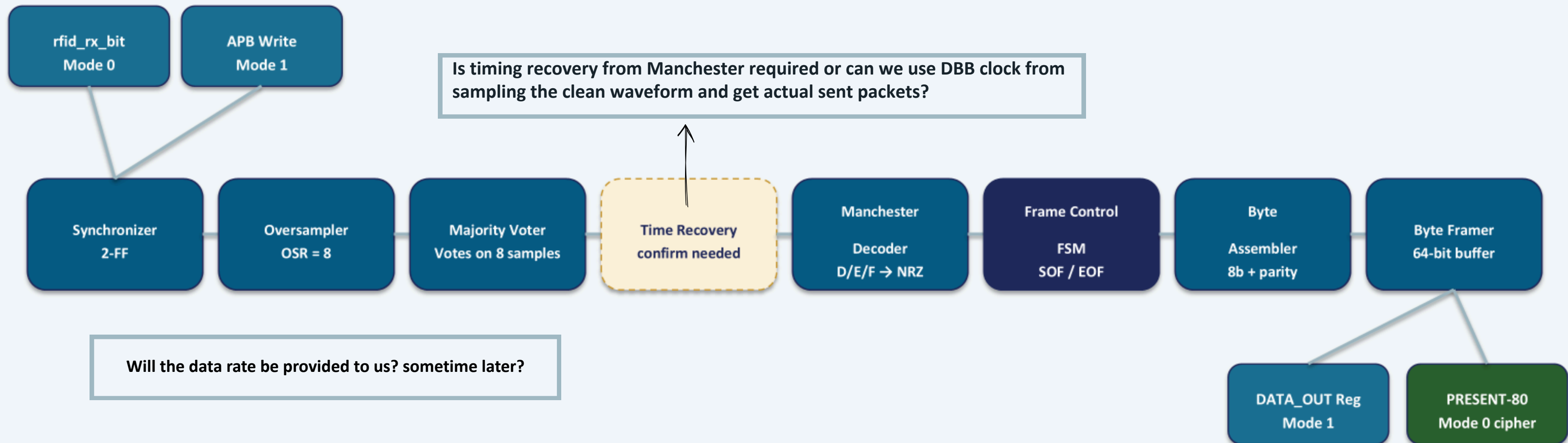
3

Are we going to use synchronous reset or asynchronous reset?

4

Is the data received from rfid always multiples of 64 bits excluding payload ? what to do if not ?

DBB Data Flow — Mode 0 & Mode 1



In MODE 1

CPU writes the data 64 bits for validating DBB block

1. 64 bit raw oversampled data - are this manchester encoded? if yes then it has payload of 32 bits?
2. By DBB block does this include complete flow mentioned above?
3. Does the data written by CPU 64 bits have preamble+CRC+parity bits included? if not how to verify Byte framer?

Doubts:

Q1 Where are we using rfid_tx_bit which is External RF interface? Are we transmitting using AFE? if yes, WHAT?

Q2 How are packets framed while transmitting? - since packet framing is protocol specific
From AFE, there are list of protocols

Q3 If we are not sure how packets framed, how to decode data from frames? Frame control FSM?

```
Top-Level Interface
module crypto_rfid_top (
    input wire    clk,
    input wire    rst_n,

    // APB Slave Interface
    input wire    psel,
    input wire    penable,
    input wire    pwrite,
    input wire [31:0] paddr,
    input wire [31:0] pwdata,
    output wire [31:0] prdata,
    output wire    pready,

    // External RF Interface
    input wire    rfid_rx_bit,
    output wire    rfid_tx_bit
);
```

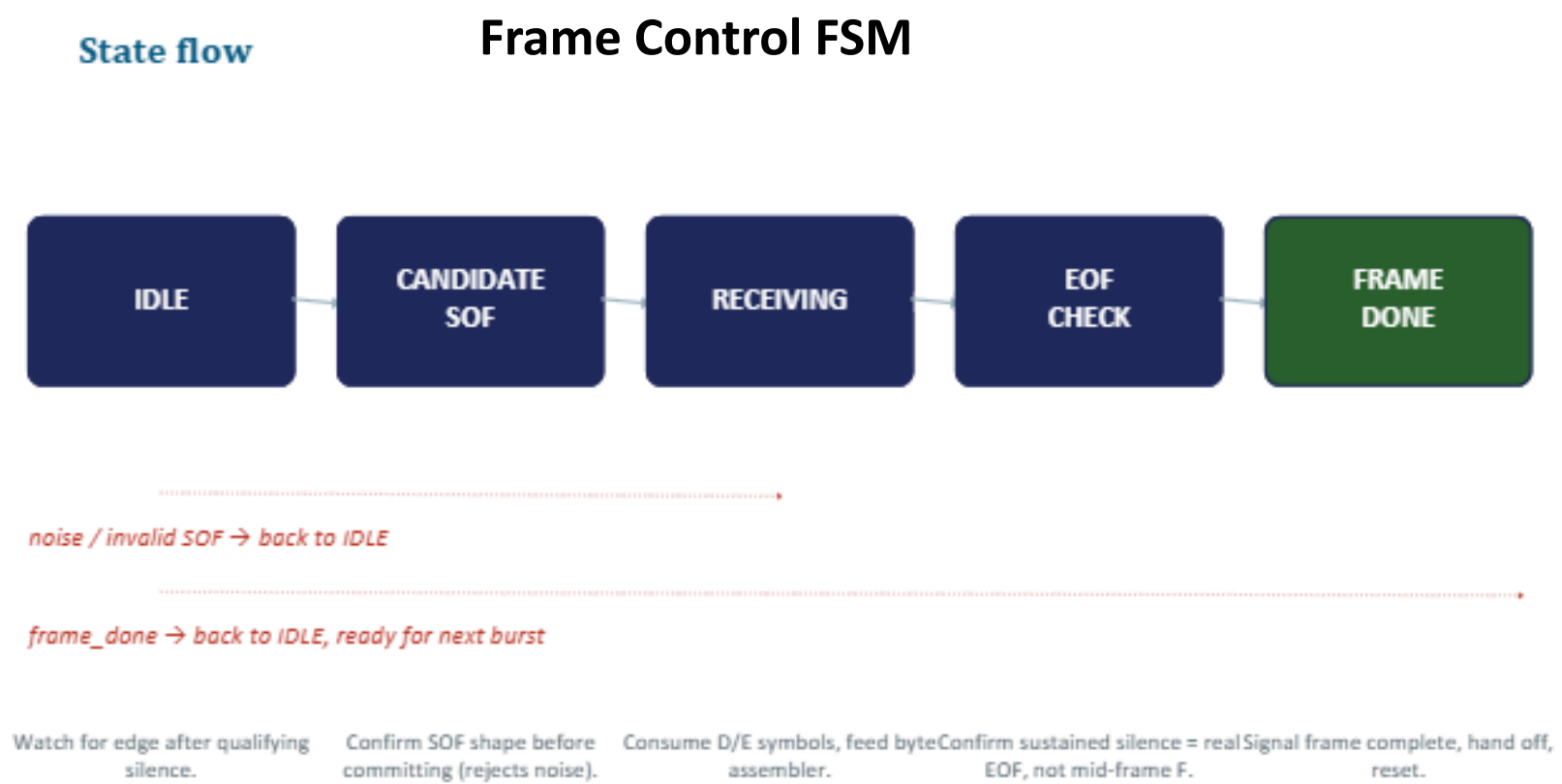
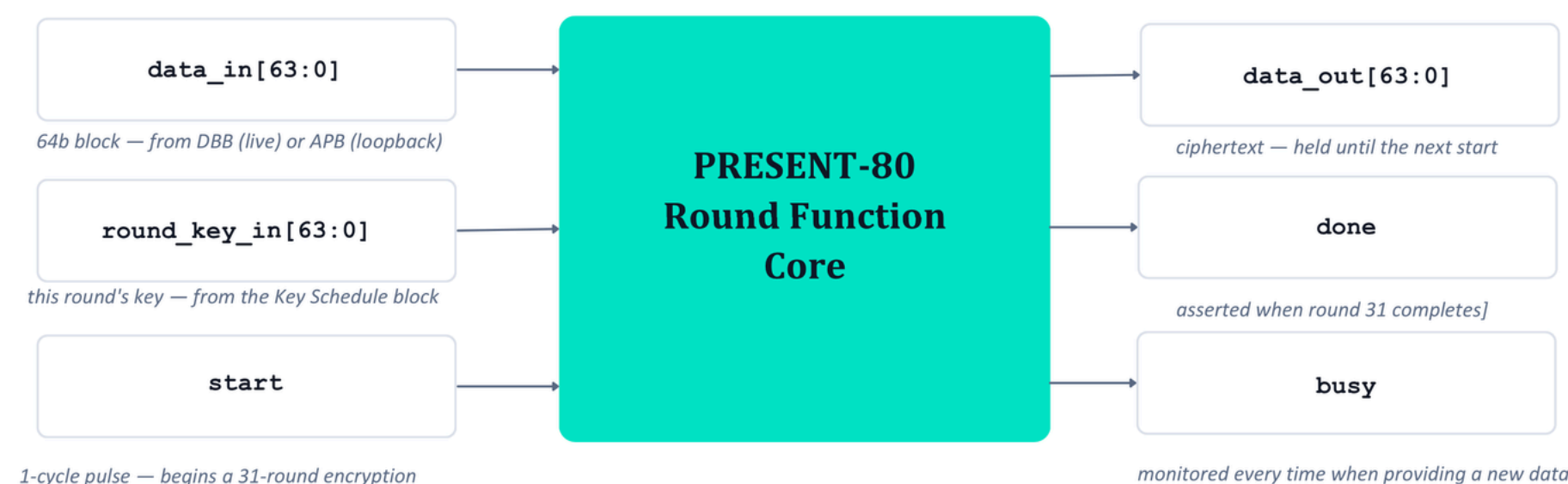


Table 3-1. Supported Modes of Operation

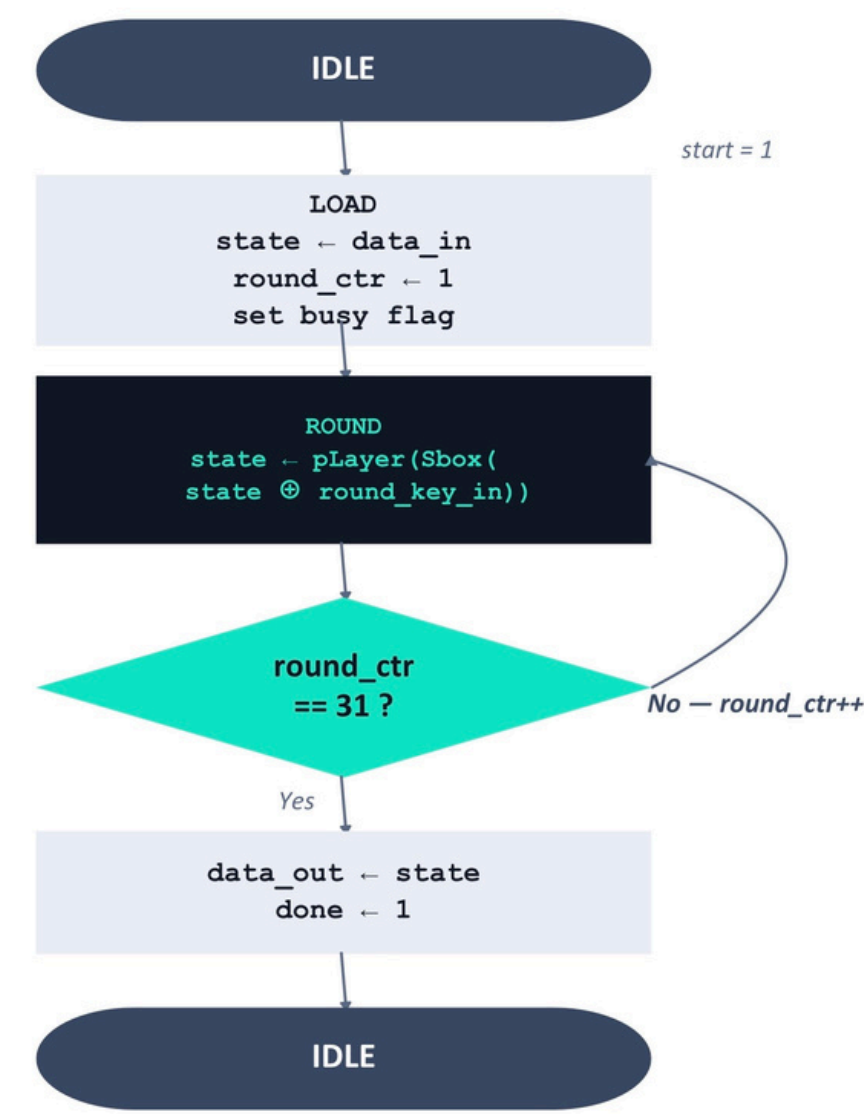
P2P INITIATOR OR READER/WRITER		CARD EMULATION		P2P TARGET	
TECHNOLOGY	BIT RATE (kbps)	TECHNOLOGY	BIT RATE (kbps)	TECHNOLOGY	BIT RATE (kbps)
NFC-A and NFC-B (ISO/IEC 14443 A and B)	106, 212, 424, 848 ⁽¹⁾	NFC-A, NFC-B	106	NFC-A	106
NFC-F (JIS: X6319-4)	212, 424	N/A	N/A	NFC-F	212, 424
NFC-V (ISO/IEC 15693)	6.7, 26.7	N/A	N/A	N/A	N/A

(1) 848 kbps applies to reader/writer mode only.

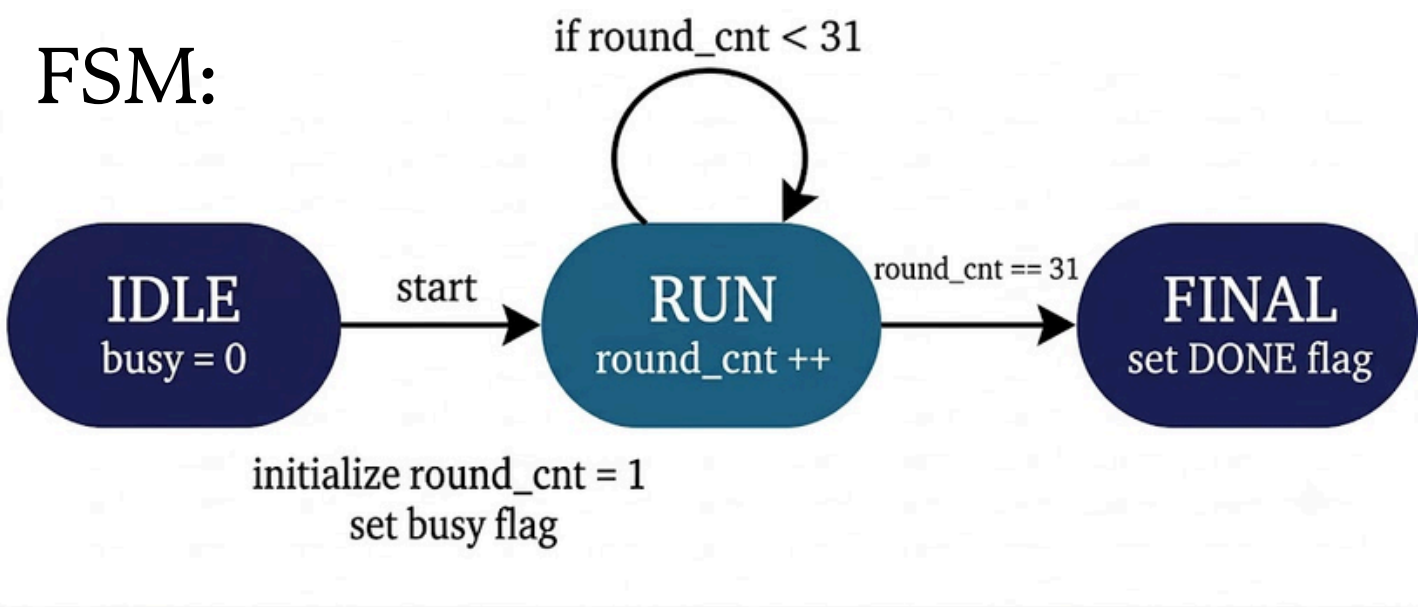
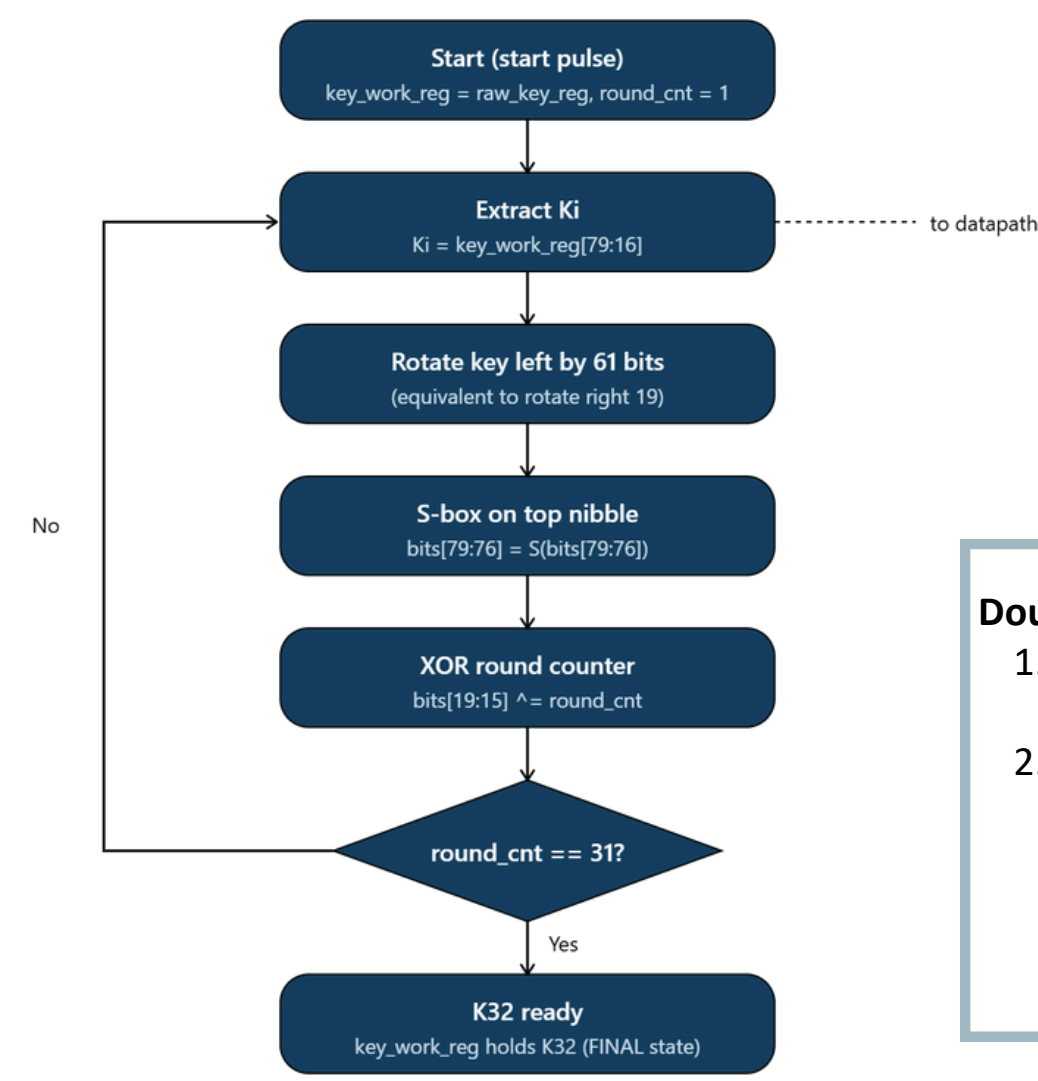
PRESENT 80 - Mode 0 & Mode 2



PRESENT-80 Core



Key Schedule Core



Doubts:

1. What about clock gating, should we write enable-gated RTL to allow synthesis tools to automatically infer Integrated Clock Gating (ICG) cells?
2. In mode 2, as we need ~32 cycles for PRESENT 80 which is long relative to how fast the CPU can issue writes (1–2 cycles), do we need a FIFO to guarantee zero data loss under bursty writes or we just assume a write while busy is undefined behaviour by spec which is handled by CPU?